

ReproZip with Docker on x86-64 Linux

What ReproZip does

ReproZip records one command and the files that command uses. It creates an `.rpz` bundle containing the program, data, libraries, environment information, and recorded command. ReproUnzip inspects the bundle and reruns the command.

```
working command -> reprozip trace -> reprozip pack -> reprounzip run
```

ReproZip does not repair a failing program. Run the original command successfully before tracing it.

Every command in these notes runs inside the course Docker image. Students do not install ReproZip, ReproUnzip, or Python packages directly on the host.

Supported hosts

ReproZip traces Linux system calls with `ptrace`. The course workflow requires an x86-64 Linux Docker host.

Host	Status
x86-64 Linux with Docker	Course test passed
Apple Silicon Mac with Docker Desktop	Course test failed during tracing
Intel Mac with Docker Desktop	Not tested
Windows with Docker Desktop or WSL	Not tested

The image built on an M1 Mac, but `reprozip trace` reported invalid emulated syscalls and crashed even when the container had `SYS_PTRACE`. Do not ask Mac students to trace through Docker Desktop's AMD64 emulation.

Students without an x86-64 Linux desktop should use the compute server assigned for the course.

Connect to the course server

Students using CIMS should connect through the access server and then move to the assigned compute server:

```
ssh <netid>@access.cims.nyu.edu
ssh <course-compute-server>
hostname
uname -s
uname -m
docker version
```

The operating system must be Linux and `uname -m` must print `x86_64`. Do not run coursework on `access.cims.nyu.edu`. The instructor must provide the compute-server hostname and confirm that Docker is available there.

If Docker is unavailable or prohibited, stop and ask course staff for the approved container runtime. Do not install a system daemon on a shared server.

Get the course repository

```
git clone https://github.com/gpu004/advance-database.git
cd advance-database
```

Run the remaining setup commands from the repository root.

Build the course image

```
docker build --platform=linux/amd64 -t reprozip:linux-x86 \
  -f reprozip/docker-x86-linux/Dockerfile reprozip/docker-x86-linux
```

Confirm the image architecture:

```
docker image inspect reprozip:linux-x86 \
  --format 'os={{.Os}} architecture={{.Architecture}}'
```

Expected result:

```
os=linux architecture=amd64
```

Start the course container

Change to the project directory that contains the program and its inputs. Then start the container:

```
docker run --rm -it --platform=linux/amd64 \
  --cap-add=SYS_PTRACE \
  -v "$PWD:/work" -w /work \
  reprozip:linux-x86
```

--cap-add=SYS_PTRACE is required for tracing. The bind mount maps the current host directory to /work, so traces and bundles remain on the host after the container exits.

The remaining commands in the required workflow run inside this container.

The required workflow

1. Test the original command

Run the program normally and inspect its output:

```
python3 demo.py input.txt output.txt
cat output.txt
```

Fix program errors before using ReproZip.

2. Trace the exact command

Prefix the working command with `reprozip trace`:

```
reprozip trace python3 demo.py input.txt output.txt
```

A successful trace creates `.reprozip-trace/config.yml`.

```
ls -la .reprozip-trace
less .reprozip-trace/config.yml
```

Before packing, inspect `config.yml` for:

- private keys, passwords, access tokens, and credentials
- licensed files that must not be distributed
- browser profiles or unrelated files
- large datasets that should not be included
- the exact command the grader should run

3. Pack the trace

```
reprozip pack project.rpz
ls -lh project.rpz
```

The `.rpz` file is the portable bundle.

4. Inspect the bundle

```
reprounzip info project.rpz
reprounzip showfiles project.rpz
```

`reprounzip info` reports the architecture, Linux distribution, recorded command, software packages, and compatible unpackers.

`reprounzip showfiles` reports named input and output files that can be replaced or downloaded.

5. Test the bundle in a fresh directory

Do not test only beside the original program.

```
mkdir -p bundle-check
reprounzip directory setup project.rpz bundle-check/unpacked
reprounzip directory run bundle-check/unpacked
```

A successful run exits with status 0. Inspect the reproduced output under `bundle-check/unpacked/root/work/`.

Multiple runs and test cases

Students may trace several runs:

```
reprozip trace --overwrite python3 main.py input1.txt output1.txt
reprozip trace --continue python3 main.py input2.txt output2.txt
reprozip pack project-tests.rpz
```

Another option is to trace one script that runs every required test:

```
chmod +x run_tests.sh
./run_tests.sh
reprozip trace ./run_tests.sh
reprozip pack project-tests.rpz
```

The traced command should exercise every file and dependency the grader needs.

Common project commands

Python

```
reprozip trace python3 main.py input.txt output.txt
```

Java

The course image does not include Java. Extend the Dockerfile with the required pinned JDK before tracing a Java project. Do not install Java interactively in a running container because that change disappears when the container exits.

C or C++

The course image does not include a compiler. Extend the Dockerfile with the required pinned compiler before tracing a C or C++ project.

Shell script

```
chmod +x run.sh
./run.sh
reprozip trace ./run.sh
```

Final-project bundle checklist

- The original program runs inside the course image on an x86-64 Linux Docker host.
- The traced command covers every required test or input.

- `.reprozip-trace/config.yml` contains no credentials or unrelated personal files.
- The `.rpz` filename identifies the project or team.
- `reprounzip info` reports the intended command and architecture.
- `reprounzip directory setup` succeeds in a fresh directory.
- `reprounzip directory run` exits with status 0.
- The reproduced output matches the expected output.
- The submission README states the command, expected result, language version, and assumptions.

Troubleshooting

Docker server is unavailable

If `docker version` cannot connect to the daemon, ask course staff which x86-64 Linux host provides the approved Docker service.

Wrong host architecture

```
uname -m
```

Use an x86-64 Linux host. Do not use Apple Silicon AMD64 emulation for ReproZip tracing.

Trace reports a permission error

Exit the container and confirm that the `docker run` command includes:

```
--cap-add=SYS_PTRACE
```

If tracing still fails, record the hostname, kernel, Docker version, exact command, ReproZip version, and complete error. Ask course staff whether `ptrace` is permitted on that host.

Bundle is unexpectedly large

Inspect `.reprozip-trace/config.yml`, remove unrelated files from the configuration, and trace again if needed.

Reproduced run misses a file

Test in a fresh directory. Retrace while the original program opens every required file.

Verified reference result

The complete workflow passed on x86-64 Debian 12 through Modal on August 30, 2026 with Python 3.11.14, ReproZip 1.3.2, and ReproUnzip 1.3.2. The trace and reproduced run both produced `REPROZIP ON NYU LINUX`.

The same trace failed under Docker Desktop's AMD64 emulation on an Apple M1 Mac. That failure is why these instructions require an x86-64 Linux host.

References

- ReproZip quickstart: <https://github.com/VIDA-NYU/reprozip#quickstart>
- ReproZip packing: <https://reprozip.readthedocs.io/en/latest/packing.html>
- ReproUnzip: <https://reprozip.readthedocs.io/en/latest/unpacking.html>
- Course repository: <https://github.com/gpu004/advance-database>
- NYU CIMS access servers: <https://cims.nyu.edu/dynamic/systems/resources/accessservers/>